

Vidya Academy of Science & Technology

(A unit of Vidya International Charitable Trust)

Approved by AICTE & Affiliated to APJ Abdul Kalam Technological University



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

CSL204 OPERATING SYSTEMS LAB

LAB MANUAL

PREPARED BY:

Ms. ATHULYA BHASKAR K R

APPROVED BY:

HOD, CSE

VISION OF THE INSTITUTION

Progress through education

MISSION OF THE INSTITUTION

To seek, strive for and scale greater heights of quality education

VISION OF THE DEPARTMENT

Progress through quality education and become a centre of excellence in computing knowledge

MISSION OF THE DEPARTMENT

To seek, to strive for advancements in technological and social aspects so as to create socially responsible engineers in the field of computer science, who will embark to further reduce gap between science and man-kind.

Program Educational Objectives (PEOs)

- PEO-1: Graduates shall practice Computer Science and Engineering in IT industry, public sector undertaking, as an entrepreneur or as a researcher pursuing a successful professional career.
- PEO-2: Graduates shall demonstrate creativity, develop innovative ideas and work in teams to accomplish a common goal.
- PEO-3: Graduates shall demonstrate project management skills, lifelong learning and social awareness upholding ethical values.

Program Specific Outcomes (psos)

- PSO-1: The ability to apply the knowledge of computational structures and algorithms for solving complex engineering problems in multidisciplinary areas.
- PSO-2: The ability to design, develop and evaluate computer based systems using advanced tools and technologies to meet the real world challenges.

Program Outcomes (POs)

Computer Science & Engineering graduates will be able to:

- PO-1: Engineering knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and Computer Science & Engineering specialization to the solution of complex engineering problems.
- PO-2: Problem analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
- PO-3: Design/development of solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
- PO-4: Conduct investigations of complex problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
- PO-5: Modern tool usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.
- PO-6: The engineer and society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.
- PO-7: Environment and sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.
- PO-8: Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
- PO-9: Individual and team work: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary surroundings.
- PO-10: Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

PO-11: Project management and finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

PO-12: Life-long learning: Recognize the need for, and have the preparation and ability to engage in Independent and life-long learning in the broadest context of technological change.

Program: B. Tech	Department: Computer Science and Engineering
Course: Operating system lab	Semester: S4 Batch: C Credits: 2
Course Code: CSL 204	Regulation: 2019
Course Area/Domain: Systems and Cyber security	L: 0 T: 0 TW/SL: 1 P: 3
Corresponding Lab Course Code (If Any): NA	Lab Course Name: NA

COURSE PRE-REQUISITES:

CODE	COURSE NAME	DESCRIPTION	SEM
CST201	Data Structures	Basics of Data Structures	S3
EST 102	Programming in C	Programming concepts in C language	S2

COURSE OUTCOMES:

SL NO	DESCRIPTION
CO1	Illustrate the use of system calls in Operating Systems
CO2	Implement process creation and inter-process communication in Operating Systems
CO3	Implement First Come First Served, Shortest Job First, Round Robin and Priority-based CPU Scheduling algorithms
CO4	Illustrate the performance of First In First Out, Least Recently Used and Least Frequently Used Page Replacement algorithms
CO5	Implement modules for deadlock detection and deadlock avoidance in Operating Systems, and implement storage management and disk scheduling algorithms

CO - PO/PSO MAPPING:

CO Number	PO STRENGTH												PSO STRENGTH	
	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO10	PO11	PO12	PSO1	PSO2
CO1	3	1	3					2		2		3		3
CO2	3	3	3					2		2		3	2	3
CO3	3	3	3	2				2		2		3	3	3
CO4	3	3	3	2				2		2		3	3	3
CO5	3	3	3	2				2		2		3	3	1

SYLLABUS

* mandatory

1. Basic Linux commands
2. Shell programming -Command syntax -Write simple functions with basic tests, loops, patterns
3. System calls of Linux operating system:* fork, exec, getpid, exit, wait, close, stat, opendir, readdir
4. Write programs using the I/O system calls of Linux operating system (open, read write)
5. Implement programs for Inter Process Communication using Shared Memory *
6. Implement Semaphores*
7. Implementation of CPU scheduling algorithms. a) Round Robin b) SJF c) FCFS d) Priority *
8. Implementation of the Memory Allocation Methods for fixed partition* a) First Fit b) Worst Fit c) Best Fit
9. Implement l page replacement algorithms a) FIFO b) LRU c) LFU*
10. Implement the banker's algorithm for deadlock avoidance. *
11. Implementation of Deadlock detection algorithm

12. Simulate file allocation strategies. a) Sequential b) Indexed c) Linked

13. Simulate disk scheduling algorithms. * a) FCFS b)SCAN c) C-SCAN

GENERAL INSTRUCTIONS

Preparation:

- Ensure you have access to a Linux system (or a Linux terminal/VM) with the GNU C compiler (gcc) installed.
- Familiarize yourself with the text editor or IDE you will be using (e.g., gedit, vi/vim, nano, Visual Studio Code).
- Make sure any required libraries are available (e.g., the POSIX thread library, linked using `-lpthread`, or the real-time library, linked using `-lrt` for semaphores).

Lab Setup:

- Create a new directory for each lab session (e.g., `mkdir exp6`) to keep your programs organized.
- Open your editor and create a new C/shell script file for each exercise, using meaningful file names.
- Ensure your terminal is in the correct working directory before compiling and running a program.

Documentation:

- Start each program with comments that include your name, date, experiment number and a brief description of what the program does.
- Write clear comments throughout your code, especially before system calls, to explain the purpose of each section.

Coding Standards:

- Follow standard C coding conventions for naming, indentation and brace placement.
- Use meaningful variable names that convey the purpose of the variable (e.g., `pid`, `shmid`, `bufferSize`) rather than single arbitrary letters.
- Check the return value of every system call (`fork`, `exec`, `open`, `read`, `write`, `shmget`, `semget`, etc.) and handle errors appropriately.

Testing and Debugging:

- Compile your code frequently as you write it, using `gcc program.c -o program`, and fix warnings/errors before proceeding.

- Use printf statements, perror() or debugging tools such as gdb to trace execution and identify issues.
- Test your program with different inputs, including boundary and invalid cases, to ensure it behaves correctly.
- Always release resources you have created — terminate child processes, close file descriptors, and remove shared memory/semaphores (ipcs, ipcrm) — before ending the program.

Submission:

- Follow the submission guidelines provided by your instructor for the observation book and record note book.
- Ensure your program compiles without errors, runs correctly, and the output has been verified before submission.
- Get the algorithm, program and output corrected and signed by the staff-in-charge in the same lab session.

Collaboration:

- Discuss approaches and logic with your peers if permitted, but ensure that the code you submit is written and understood by you.
- Discuss different ways of solving a problem (e.g., different scheduling or page replacement strategies) and share insights.

Resources:

- Utilize online resources such as man pages (man 2 fork, man 3 printf), official documentation and forums (e.g., Stack Overflow) if you encounter difficulties.
- Refer to the Linux manual pages and the C standard library documentation for the syntax of system calls and library functions.

Ask for Help:

- Don't hesitate to ask your instructor or lab assistant for help if you get stuck, encounter an unfamiliar error, or need clarification on a task.

BEST PRACTICES**Understand the Basics:**

- Ensure you have a solid grasp of C programming basics — syntax, data types, pointers, functions and file handling — before attempting system-call based programs.

Plan Your Code:

- Before you start coding, write down the algorithm/procedure for the experiment. Outline the steps needed and think about how the program will be organized.

Write Readable Code:

- Follow consistent indentation, spacing and naming conventions throughout your program.
- Use comments to explain the purpose of important sections, especially around system calls and synchronization logic.

Modular Programming:

- Break your code into smaller, reusable functions wherever possible (e.g., separate functions for FCFS, SJF and Priority scheduling logic).
- Each function should ideally have a single, well-defined responsibility.

Testing:

- Test your program regularly as you write it, rather than writing the entire program before the first compilation.
- Consider edge cases (e.g., empty buffers, zero processes, insufficient memory) and handle potential errors using proper checks.

Use System Resources Responsibly:

- Be mindful when using system calls that create processes, shared memory, semaphores or files — always clean up resources you create.
- Avoid infinite loops or unbounded process creation (fork bombs), which can affect the entire system.

Documentation:

- Maintain clear documentation for each experiment in your record: aim, algorithm, program and result.
- Ensure the expected output noted in the record matches the actual output obtained on execution.

Stay Updated:

- Keep up with basic Linux/OS concepts covered in theory, as they form the foundation for the lab exercises.

Ask Questions:

- If you encounter difficulties or have questions about a concept or error, don't hesitate to ask for help from peers or the instructor.

Practice Regularly:

- The more you practice writing and debugging C programs on Linux, the more comfortable you will become with system programming concepts.

Collaborate and Share:

- Work with others to understand different approaches, but write and test your own code independently.

Follow Ethical Guidelines:

- Do not copy programs or results from other students or unauthorized sources. Ensure the work you submit is your own.

INDEX

Exp No.	Expt. Name	Page Number
1	BASIC LINUX COMMANDS	10
2	SHELL PROGRAMMING	13
3	SYSTEM CALLS OF LINUX OPERATING SYSTEM	17
4	LINUX I/O SYSTEM CALLS (OPEN, READ, WRITE)	21
5	INTER-PROCESS COMMUNICATION (IPC) USING SHARED MEMORY	25
6	PROCESS SYNCHRONIZATION USING SEMAPHORES	28
7	IMPLEMENTATION OF CPU SCHEDULING ALGORITHMS	32
8	IMPLEMENTATION OF MEMORY ALLOCATION METHODS	36
9	IMPLEMENTATION OF PAGE REPLACEMENT ALGORITHMS	40
10	IMPLEMENTATION OF BANKER'S ALGORITHM FOR DEADLOCK AVOIDANCE	43
11	IMPLEMENTATION OF DEADLOCK DETECTION ALGORITHM	47
12	SIMULATION OF FILE ALLOCATION STRATEGIES	51
13	SIMULATION OF DISK SCHEDULING ALGORITHMS	55

EXP NO: 1**STUDY OF BASIC LINUX COMMANDS****AIM**

To study and practice basic Linux commands related to file handling, directory management, process management and general system administration.

LEARNING OUTCOMES

- Understand the structure of the Linux file system and directory hierarchy.
- Learn commands for creating, viewing, copying, moving and deleting files and directories.
- Learn commands for viewing and managing running processes.
- Learn commands for checking system information, permissions and user administration.

DESCRIPTION

Linux is a multi-user, multitasking operating system that provides a command-line interface (shell) through which the user can interact with the system. Basic Linux commands allow the user to navigate the file system, create and manipulate files and directories, change file permissions, monitor and control processes, and obtain system information. Mastery of these commands forms the foundation for shell programming and systems programming on Linux.

ALGORITHM

1. Open a terminal window on the Linux system.
2. Practice file and directory commands: pwd, ls, cd, mkdir, rmdir, touch, cp, mv, rm, cat.
3. Practice permission commands: chmod, chown, ls -l.
4. Practice process management commands: ps, top, kill, jobs, bg, fg.
5. Practice system information commands: who, whoami, date, uname -a, df, du, man.
6. Note down the purpose and output of each command executed.

PROCEDURE

1. Open a terminal in the Linux system.
2. Execute file/directory commands one by one, e.g., pwd, ls -l, mkdir testdir, cd testdir, touch file1.txt.

3. Execute cp, mv and rm commands to copy, rename/move and delete files.
4. Execute chmod 755 file1.txt to change file permissions and verify using ls -l.
5. Execute process commands such as ps -ef and top to view running processes; use kill <pid> to terminate a process.
6. Execute system information commands such as who, date, uname -a and df -h.
7. Record the command, its syntax and the observed output for each command.

PROGRAM

--- File and Directory Commands ---

```
pwd                # print current working directory
ls -l              # list files with details
mkdir testdir      # create a directory
cd testdir         # change directory
touch file1.txt    # create an empty file
cp file1.txt file2.txt # copy a file
mv file2.txt file3.txt # rename/move a file
rm file3.txt       # delete a file
cat > file1.txt    # create/edit file content
cat file1.txt      # display file content
```

--- Permission Commands ---

```
chmod 755 file1.txt # change file permissions
chown user1 file1.txt # change file owner
ls -l file1.txt     # verify permissions
```

--- Process Management Commands ---

```
ps -ef            # list all running processes
top               # dynamic view of processes
kill -9 1234      # terminate process with PID 1234
jobs              # list background jobs
bg                # resume a job in background
fg                # bring a job to foreground
```

--- System Information Commands ---

```
who                # list logged-in users
whoami             # display current user
date               # display current date and time
uname -a           # display system information
df -h              # display disk usage
du -sh testdir     # display directory size
man ls             # display manual page for ls
```

EXPECTED OUTPUT

```
$ pwd
/home/student
$ mkdir testdir
$ ls -l
drwxr-xr-x 2 student student 4096 Jul 28 10:00 testdir
$ chmod 755 file1.txt
$ ls -l file1.txt
-rwxr-xr-x 1 student student 0 Jul 28 10:02 file1.txt
$ ps -ef | head -3
UID  PID  PPID  C  STIME TTY  TIME   CMD
root  1    0  0 09:00 ?    00:00:02 /sbin/init
student 512  1  0 09:01 ?    00:00:00 bash
$ uname -a
Linux ubuntu 6.8.0-generic x86_64 GNU/Linux
```

MODEL VIVA QUESTIONS

1. What is the difference between an absolute path and a relative path?
2. What is the difference between the rm and rmdir commands?
3. What do the permission bits in the output of ls -l represent?
4. What is the difference between the ps and top commands?
5. How do you send a specific signal to a process using the kill command?

RESULT

Basic Linux commands for file handling, permissions, process management and system information were studied and practiced successfully.

EXPNO: 2**SHELL PROGRAMMING - COMMAND SYNTAX****AIM**

To write simple shell scripts using basic tests, loops and patterns to solve simple problems.

LEARNING OUTCOMES

- Understand the syntax and structure of a shell script and the role of the shebang line.
- Learn to use variables, command-line arguments and basic input/output in shell scripts.
- Learn to use conditional statements (if, case) and test operators in shell scripts.
- Learn to use loop constructs (for, while, until) and pattern matching in shell scripts.

DESCRIPTION

Shell programming (shell scripting) allows a sequence of Linux commands to be combined into a single script that can be executed as a program, enabling automation of repetitive tasks. A shell script begins with a shebang line (e.g., `#!/bin/bash`) that specifies the interpreter to be used. Shell scripts support variables, command-line arguments, standard input/output, control structures such as if-else and case for decision making, loop constructs such as for, while and until for repetition, and pattern matching using wildcards and the case statement.

ALGORITHM

1. Start the script with the shebang line `#!/bin/bash`.
2. Declare variables and, if required, read command-line arguments (`$1`, `$2`, ...) or user input using the `read` command.
3. Use conditional statements (if...then...else...fi or case...esac) along with test operators (`-eq`, `-lt`, `-gt`, `-f`, `-d`, etc.) to make decisions.
4. Use loop constructs (for, while, until) to repeat a set of statements as required.
5. Display the result using the `echo` command.
6. Save the script, give it execute permission and run it.

PROCEDURE

1. Open a terminal in the Linux system.
2. Create a file: gedit script.sh
3. Type the shell script given below and save the file.
4. Give execute permission using: chmod +x script.sh
5. Execute the script using: ./script.sh or bash script.sh
6. Observe the output and verify the logic for different inputs.

PROGRAM

```
#!/bin/bash
# Shell script demonstrating basic tests, loops and patterns

echo "Enter a number:"
read num

# --- if-else test ---
if [ $num -gt 0 ]; then
    echo "$num is Positive"
elif [ $num -lt 0 ]; then
    echo "$num is Negative"
else
    echo "$num is Zero"
fi

# --- for loop: print first n natural numbers ---
echo "Enter the value of n:"
read n
echo "Natural numbers from 1 to $n:"
for (( i=1; i<=n; i++ ))
do
    echo -n "$i "
done
```

```
echo

# --- while loop: sum of digits ---
temp=$num
sum=0
while [ $temp -ne 0 ]
do
    digit=$(( temp % 10 ))
    sum=$(( sum + digit ))
    temp=$(( temp / 10 ))
done
echo "Sum of digits of $num = $sum"

# --- case / pattern matching: check file type ---
echo "Enter a filename:"
read fname
case $fname in
    *.txt) echo "$fname is a text file" ;;
    *.c)  echo "$fname is a C program file" ;;
    *.sh) echo "$fname is a shell script" ;;
    *)    echo "Unknown file type" ;;
esac
```

EXPECTED OUTPUT

```
Enter a number:
25
25 is Positive
Enter the value of n:
5
Natural numbers from 1 to 5:
1 2 3 4 5
Sum of digits of 25 = 7
Enter a filename:
report.txt
report.txt is a text file
```

MODEL VIVA QUESTIONS

1. What is the purpose of the shebang line (`#!/bin/bash`) in a shell script?
2. What is the difference between the `sh` and `bash` shells?
3. How do you make a shell script executable?
4. What is the difference between the `test` command and `[ ]` in shell scripting?
5. Differentiate between the `for`, `while` and `until` loops.

RESULT

Shell scripts demonstrating basic tests, loops and pattern matching were written and executed successfully.

EXPNO: 3**SYSTEM CALLS OF LINUX OPERATING SYSTEM****AIM**

To write C programs to study and demonstrate the Linux system calls: fork, exec, getpid, exit, wait, close, stat, opendir and readdir.

LEARNING OUTCOMES

- Understand the concept of a system call and the transition between user mode and kernel mode.
- Learn the working of process creation and replacement using fork() and exec().
- Learn the working of process identification and termination using getpid(), exit() and wait().
- Learn the working of file and directory related system calls: close(), stat(), opendir() and readdir().

DESCRIPTION

A system call is the mechanism through which a user-level process requests a service from the operating system kernel, such as process control, file manipulation or device management. The fork() system call creates a new process (child) as a copy of the calling process (parent). The exec() family of system calls replaces the current process image with a new program. getpid() returns the process identifier of the calling process. exit() terminates a process, and wait() causes the parent process to suspend execution until a child process terminates. close() closes an open file descriptor, stat() retrieves information about a file (such as size, permissions and modification time), and opendir()/readdir() are used to open a directory stream and read directory entries respectively.

ALGORITHM

1. Start the program and call fork() to create a child process.
2. In the child process, call getpid() to display its own process ID and, optionally, call exec() to replace its image with another program.
3. In the parent process, call wait() to wait for the child process to terminate, and then display a message.
4. Call exit() to terminate a process with a specific exit status.
5. In a separate program, call open()/stat() to retrieve and display file information such as size and permissions.

6. In a separate program, call opendir() to open a directory and use readdir() in a loop to list all directory entries, then call closedir()/close() to release resources.
7. Stop the program.

PROCEDURE

1. Open a terminal in the Linux system.
2. Create a file: gedit syscalls.c
3. Type the C program given below and save the file.
4. Compile using: gcc syscalls.c -o syscalls
5. Execute using: ./syscalls
6. Observe the process IDs, exit status, file information and directory listing displayed.

PROGRAM

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <sys/stat.h>
#include <dirent.h>
int main() {
    pid_t pid;
    printf("Parent process ID (before fork) = %d\n", getpid());
    pid = fork();          /* fork() system call */
    if (pid < 0) {
        printf("Fork failed\n");
        exit(1);
    } else if (pid == 0) {
        /* Child process */
        printf("Child process, PID = %d, Parent PID = %d\n", getpid(), getppid());
        execlp("/bin/ls", "ls", "-l", NULL); /* exec() system call */
        printf("exec failed\n");
        exit(1);
    } else {
        /* Parent process */
```

```
int status;
wait(&status);      /* wait() system call */
printf("Parent process, PID = %d, Child terminated\n", getpid());
}
/* --- stat() system call --- */
struct stat fileInfo;
if (stat("syscalls.c", &fileInfo) == 0) {
    printf("\nFile: syscalls.c\n");
    printf("Size: %ld bytes\n", fileInfo.st_size);
    printf("Permissions: %o\n", fileInfo.st_mode & 0777);
}
/* --- opendir() and readdir() system calls --- */
DIR *dir = opendir(".");
struct dirent *entry;
printf("\nDirectory listing:\n");
if (dir != NULL) {
    while ((entry = readdir(dir)) != NULL) {
        printf("%s\n", entry->d_name);
    }
    closedir(dir);
}
exit(0);             /* exit() system call */
}
```

EXPECTED OUTPUT

Parent process ID (before fork) = 4210

Child process, PID = 4211, Parent PID = 4210

total 24

-rwxr-xr-x 1 student student 8472 Jul 28 10:20 syscalls

-rw-r--r-- 1 student student 1523 Jul 28 10:19 syscalls.c

Parent process, PID = 4210, Child terminated

File: syscalls.c

Size: 1523 bytes

Permissions: 644

Directory listing:

.

..

syscalls

syscalls.c

MODEL VIVA QUESTIONS

1. What is a system call and how is it different from a library function?
2. What is the difference between fork() and exec()?
3. What does fork() return in the parent process and in the child process?
4. What is the purpose of the wait() system call?
5. What is a zombie process and an orphan process?

RESULT

C programs demonstrating the Linux system calls fork, exec, getpid, exit, wait, close, stat, opendir and readdir were executed successfully.

EXPNO: 4**PROGRAMS USING I/O SYSTEM CALLS OF LINUX OPERATING SYSTEM****AIM**

To write C programs using the I/O system calls of the Linux operating system: open, read and write.

LEARNING OUTCOMES

- Understand the concept of file descriptors and low-level (unbuffered) I/O in Linux.
- Learn the working and syntax of the open(), read(), write() and close() system calls.
- Implement a C program to create/open a file and write data into it.
- Implement a C program to read data from a file and copy the contents of one file into another.

DESCRIPTION

Unlike the buffered I/O functions of the C standard library (such as fopen, fread, fwrite), the open(), read() and write() functions are low-level Linux system calls that operate directly on file descriptors and perform unbuffered I/O. The open() system call opens or creates a file and returns a file descriptor, which is an integer handle used in subsequent read() and write() calls. The read() system call reads a specified number of bytes from a file descriptor into a buffer, and the write() system call writes a specified number of bytes from a buffer to a file descriptor. The close() system call releases the file descriptor when it is no longer needed.

ALGORITHM

1. Start the program and use open() with appropriate flags (O_CREAT, O_WRONLY, O_RDONLY, etc.) and permission mode to open/create the source and destination files.
2. Check the file descriptor returned by open() for errors (a return value of -1 indicates failure).
3. Use read() in a loop to read a block of bytes from the source file descriptor into a buffer.
4. Use write() to write the bytes read from the buffer to the destination file descriptor.
5. Repeat steps 3 and 4 until read() returns 0 (end of file).
6. Use close() to close all open file descriptors.
7. Stop the program.

PROCEDURE

1. Open a terminal in the Linux system.
2. Create a file: gedit filecopy.c
3. Type the C program given below and save the file.
4. Compile using: gcc filecopy.c -o filecopy
5. Create a sample source file, e.g.: echo 'Hello Linux I/O System Calls' > source.txt
6. Execute using: ./filecopy source.txt destination.txt and verify the contents of destination.txt using cat destination.txt

PROGRAM

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#define BUF_SIZE 1024
int main(int argc, char *argv[]) {
    int src_fd, dest_fd;
    ssize_t bytesRead;
    char buffer[BUF_SIZE];
    if (argc != 3) {
        printf("Usage: %s <source_file> <destination_file>\n", argv[0]);
        exit(1);
    }
    /* open() system call - open source file for reading */
    src_fd = open(argv[1], O_RDONLY);
    if (src_fd < 0) {
        perror("Error opening source file");
        exit(1);
    }
    /* open() system call - create/open destination file for writing */
    dest_fd = open(argv[2], O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (dest_fd < 0) {
        perror("Error opening destination file");
        close(src_fd);
```

```
    exit(1);
}
/* read() and write() system calls */
while ((bytesRead = read(src_fd, buffer, BUF_SIZE)) > 0) {
    if (write(dest_fd, buffer, bytesRead) != bytesRead) {
        perror("Error writing to destination file");
        exit(1);
    }
}
printf("File copied successfully from %s to %s\n", argv[1], argv[2]);
/* close() system call */
close(src_fd);
close(dest_fd);
return 0;
}
```

EXPECTED OUTPUT

\$ echo 'Hello Linux I/O System Calls' > source.txt

\$./filecopy source.txt destination.txt

File copied successfully from source.txt to destination.txt

\$ cat destination.txt

Hello Linux I/O System Calls

MODEL VIVA QUESTIONS

1. What is a file descriptor?
2. What are the default file descriptors 0, 1 and 2 in Linux?
3. What is the difference between open()/read()/write() and fopen()/fread()/fwrite()?
4. What are the common flags used with the open() system call (O_RDONLY, O_WRONLY, O_CREAT, O_TRUNC)?
5. What does the read() system call return when the end of file is reached?

RESULT

A C program using the Linux I/O system calls open, read and write to copy the contents of one file to another was executed successfully.

EXPNO: 5

IMPLEMENTATION OF INTER PROCESS COMMUNICATION USING SHARED MEMORY

AIM

To write a C program to implement Inter Process Communication (IPC) between two processes using Shared Memory.

LEARNING OUTCOMES

- Understand the concept of Inter Process Communication and its need in a multiprocessing environment.
- Learn the working of shared memory as an IPC mechanism.
- Implement System V shared memory system calls: shmget(), shmat(), shmdt() and shmctl().
- Implement a writer process and a reader process that communicate through a common shared memory segment.

DESCRIPTION

Inter Process Communication (IPC) refers to the mechanisms provided by the operating system that allow processes to communicate and synchronize their actions. Shared memory is one of the fastest IPC mechanisms because it allows two or more processes to access a common region of memory directly, without the overhead of copying data through the kernel (as required by pipes or message queues). In Linux, System V shared memory is implemented using shmget() to create/access a shared memory segment, shmat() to attach the segment to a process's address space, shmdt() to detach the segment, and shmctl() to control/remove the segment. Since shared memory itself does not provide synchronization, processes accessing it must use additional mechanisms (such as semaphores) to avoid race conditions in a real application.

ALGORITHM

1. Writer process: generate a unique key using ftok() and create a shared memory segment using shmget().
2. Attach the shared memory segment to the process's address space using shmat().
3. Write data into the shared memory segment.
4. Detach the shared memory segment using shmdt().

5. Reader process: obtain the same shared memory segment using `shmget()` with the same key.
6. Attach the segment using `shmat()`, read the data written by the writer process, and display it.
7. Detach the segment using `shmdt()` and remove the shared memory segment using `shmctl()` with the `IPC_RMID` command.
8. Stop the program.

PROCEDURE

1. Open a terminal in the Linux system.
2. Create two files: `gedit writer.c` and `gedit reader.c`
3. Type the writer program and the reader program given below and save both files.
4. Compile both programs using: `gcc writer.c -o writer` and `gcc reader.c -o reader`
5. Execute the writer program first using: `./writer` and enter the message to be written to shared memory.
6. Execute the reader program using: `./reader` to read and display the message from shared memory.

PROGRAM

```
/* writer.c - writes data into shared memory */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/ipc.h>
#include <sys/shm.h>
int main() {
    key_t key = ftok("shmfile", 65);
    int shmid = shmget(key, 1024, 0666 | IPC_CREAT);
    char *str = (char *) shmat(shmid, NULL, 0);
    printf("Enter data to write into shared memory: ");
    fgets(str, 1024, stdin);
    printf("Data written to shared memory: %s\n", str);
    shmdt(str);      /* detach shared memory */
    return 0;
}

/* reader.c - reads data from shared memory */
```

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/ipc.h>
#include <sys/shm.h>
int main() {
    key_t key = ftok("shmfile", 65);
    int shmid = shmget(key, 1024, 0666 | IPC_CREAT);
    char *str = (char *) shmat(shmid, NULL, 0);
    printf("Data read from shared memory: %s\n", str);
    shmdt(str);          /* detach shared memory */
    shmctl(shmid, IPC_RMID, NULL); /* remove shared memory segment */
    return 0;
}
```

EXPECTED OUTPUT

\$./writer

Enter data to write into shared memory: Hello from Writer Process

Data written to shared memory: Hello from Writer Process

\$./reader

Data read from shared memory: Hello from Writer Process

MODEL VIVA QUESTIONS

1. What is Inter Process Communication (IPC) and why is it needed?
2. What are the different IPC mechanisms available in Linux?
3. Why is shared memory considered the fastest IPC mechanism?
4. What is the purpose of the ftok() function?
5. Explain the purpose of shmget(), shmat(), shmdt() and shmctl() system calls.

RESULT

A C program to implement Inter Process Communication between a writer process and a reader process using Shared Memory was executed successfully and the output was verified.

EXP NO: 6**IMPLEMENTATION OF SEMAPHORES****AIM**

To write a C program to implement process synchronization using semaphores and to study the producer-consumer problem.

LEARNING OUTCOMES

- Understand the concept of process synchronization and the critical section problem.
- Learn the working of counting and binary semaphores.
- Apply semaphores (wait and signal operations) to solve synchronization problems.
- Implement the classical producer-consumer problem using semaphores.

DESCRIPTION

A semaphore is an integer variable that is accessed only through two standard atomic operations: wait(P) and signal(V). Semaphores are used to control access to a common resource by multiple processes/threads and to avoid race conditions in a concurrent system. A binary semaphore can take only the values 0 and 1 and is used for mutual exclusion, while a counting semaphore can take any non-negative integer value and is used to control access to a resource with multiple instances. In this experiment, POSIX semaphores are used with threads to implement the producer-consumer problem, where the producer thread inserts items into a shared buffer and the consumer thread removes items from it, ensuring that the producer does not add data to a full buffer and the consumer does not remove data from an empty buffer.

ALGORITHM

1. Start the program and initialize a shared buffer of fixed size.
2. Initialize two semaphores: empty (initial value = buffer size) and full (initial value = 0), and a mutex semaphore (initial value = 1).
3. Create the producer thread. In a loop: wait(empty), wait(mutex); insert item into buffer; signal(mutex), signal(full).
4. Create the consumer thread. In a loop: wait(full), wait(mutex); remove item from buffer; signal(mutex), signal(empty).

5. Run the producer and consumer threads concurrently and display the items produced and consumed.
6. Join the threads and terminate the program.

PROCEDURE

1. Open a terminal in the Linux system.
2. Create a file using a text editor, for example: gedit semaphore.c
3. Type the C program given below and save the file.
4. Compile the program using the command: gcc semaphore.c -o semaphore -lpthread
5. Execute the program using the command: ./semaphore
6. Observe and note down the output produced by the producer and consumer threads.

PROGRAM

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#define BUFFER_SIZE 5
int buffer[BUFFER_SIZE];
int in = 0, out = 0;
sem_t empty, full, mutex;
void *producer(void *arg) {
    for (int i = 1; i <= 10; i++) {
        sem_wait(&empty);
        sem_wait(&mutex);
        buffer[in] = i;
        printf("Producer produced: %d\n", i);
        in = (in + 1) % BUFFER_SIZE;
        sem_post(&mutex);
        sem_post(&full);
        sleep(1);
    }
    return NULL;
}
```

```
void *consumer(void *arg) {
    for (int i = 1; i <= 10; i++) {
        sem_wait(&full);
        sem_wait(&mutex);
        int item = buffer[out];
        printf("Consumer consumed: %d\n", item);
        out = (out + 1) % BUFFER_SIZE;
        sem_post(&mutex);
        sem_post(&empty);
        sleep(2);
    }
    return NULL;
}

int main() {
    pthread_t prod, cons;
    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);
    sem_init(&mutex, 0, 1);
    pthread_create(&prod, NULL, producer, NULL);
    pthread_create(&cons, NULL, consumer, NULL);
    pthread_join(prod, NULL);
    pthread_join(cons, NULL);
    sem_destroy(&empty);
    sem_destroy(&full);
    sem_destroy(&mutex);
    return 0;
}
```

EXPECTED OUTPUT

Producer produced: 1
Consumer consumed: 1
Producer produced: 2
Producer produced: 3
Consumer consumed: 2
Producer produced: 4

...

Producer produced: 10

Consumer consumed: 10

MODEL VIVA QUESTIONS

1. What is a semaphore? Differentiate between binary and counting semaphores.
2. What are the wait() and signal() operations?
3. What is the critical section problem?
4. What is a race condition?
5. What is the producer-consumer problem?

RESULT

The C program to implement process synchronization using semaphores (producer-consumer problem) was executed successfully and the output was verified.

EXP NO: 7**IMPLEMENTATION OF CPU SCHEDULING ALGORITHMS****AIM**

To write C programs to implement CPU scheduling algorithms: (a) Round Robin (b) Shortest Job First (SJF) (c) First Come First Served (FCFS) (d) Priority Scheduling, and to compute the waiting time and turnaround time of each process.

LEARNING OUTCOMES

- Understand the concept of CPU scheduling and various scheduling criteria.
- Differentiate between preemptive and non-preemptive scheduling algorithms.
- Compute waiting time, turnaround time and average waiting/turnaround time for a set of processes.
- Compare the performance of FCFS, SJF, Priority and Round Robin scheduling algorithms.

DESCRIPTION

CPU scheduling is the process by which the operating system decides which of the processes in the ready queue is to be allocated the CPU. FCFS schedules processes in the order in which they arrive. SJF selects the process with the smallest burst time next. Priority scheduling selects the process with the highest priority (lowest priority number). Round Robin allocates the CPU to each process for a fixed time quantum in a cyclic manner, making it suitable for time-sharing systems. The performance of these algorithms is measured using metrics such as waiting time, turnaround time and average waiting time.

ALGORITHM

1. Start the program and input the number of processes n.
2. Input the arrival time, burst time (and priority, if required) of each process.
3. For FCFS: sort processes by arrival time and calculate completion, turnaround and waiting time sequentially.
4. For SJF: at every scheduling point, select the process with the shortest burst time among the arrived processes.
5. For Priority: at every scheduling point, select the process with the highest priority among the arrived processes.

6. For Round Robin: maintain a ready queue and allocate the CPU to each process for a fixed time quantum, requeueing unfinished processes.
7. Compute turnaround time = completion time – arrival time and waiting time = turnaround time – burst time for each process.
8. Compute and display the average waiting time and average turnaround time.
9. Stop the program.

PROCEDURE

1. Open a terminal in the Linux system.
2. Create separate files for each algorithm, e.g., fcfs.c, sjf.c, priority.c, rr.c
3. Type the corresponding C program and save the file.
4. Compile using: gcc fcfs.c -o fcfs (similarly for other files)
5. Execute using: ./fcfs and enter the number of processes, arrival time and burst time as prompted.
6. Note down the waiting time, turnaround time and average values displayed.

PROGRAM

```
/* Round Robin Scheduling */
#include <stdio.h>
int main() {
    int n, bt[20], wt[20], tat[20], rem[20], quantum;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    printf("Enter burst time of each process:\n");
    for (int i = 0; i < n; i++) {
        scanf("%d", &bt[i]);
        rem[i] = bt[i];
    }
    printf("Enter time quantum: ");
    scanf("%d", &quantum);
    int t = 0, done;
    do {
        done = 1;
        for (int i = 0; i < n; i++) {
```

```

        if (rem[i] > 0) {
            done = 0;
            if (rem[i] > quantum) {
                t += quantum;
                rem[i] -= quantum;
            } else {
                t += rem[i];
                wt[i] = t - bt[i];
                rem[i] = 0;
            }
        }
    }
} while (!done);

float total_wt = 0, total_tat = 0;
printf("Process\tBT\tWT\tTAT\n");
for (int i = 0; i < n; i++) {
    tat[i] = bt[i] + wt[i];
    total_wt += wt[i];
    total_tat += tat[i];
    printf("P%d\t%d\t%d\t%d\n", i + 1, bt[i], wt[i], tat[i]);
}

printf("Average Waiting Time = %.2f\n", total_wt / n);
printf("Average Turnaround Time = %.2f\n", total_tat / n);
return 0;
}

```

/* Similar programs are written for FCFS (sort by arrival time and schedule in order), SJF (select process with minimum burst time among arrived processes) and Priority Scheduling (select process with highest priority, i.e. lowest priority number, among arrived processes), following the same structure of computing waiting time, turnaround time and their averages. */

EXPECTED OUTPUT

Enter number of processes: 4

Enter burst time of each process:

5 4 2 1

Enter time quantum: 2

Process	BT	WT	TAT
---------	----	----	-----

P1	5	6	11
----	---	---	----

P2	4	7	11
----	---	---	----

P3	2	5	7
----	---	---	---

P4	1	4	5
----	---	---	---

Average Waiting Time = 5.50

Average Turnaround Time = 8.50

MODEL VIVA QUESTIONS

1. What is CPU scheduling and why is it needed?
2. Differentiate between preemptive and non-preemptive scheduling.
3. What is the difference between waiting time and turnaround time?
4. What is convoy effect and in which algorithm does it occur?
5. What is starvation? Which algorithm suffers from starvation?

RESULT

C programs to implement FCFS, SJF, Priority and Round Robin CPU scheduling algorithms were executed successfully and the waiting time, turnaround time and their averages were computed.

EXP NO: 8**IMPLEMENTATION OF MEMORY ALLOCATION METHODS FOR FIXED PARTITION****AIM**

To write a C program to implement fixed partition memory allocation methods: (a) First Fit (b) Best Fit (c) Worst Fit.

LEARNING OUTCOMES

- Understand the concept of fixed partition (contiguous) memory allocation.
- Learn the working of First Fit, Best Fit and Worst Fit allocation strategies.
- Implement and compare the three memory allocation strategies for a given set of processes and memory blocks.
- Analyze internal fragmentation resulting from each strategy.

DESCRIPTION

In fixed (contiguous) partition memory allocation, the main memory is divided into fixed-size partitions/blocks and each incoming process is allocated to a partition based on a chosen allocation strategy. First Fit allocates the first block that is large enough to hold the process. Best Fit allocates the smallest block among those that are large enough, thereby minimizing wasted space. Worst Fit allocates the largest available block, leaving the largest possible leftover space. Each strategy affects the degree of internal fragmentation differently.

ALGORITHM

1. Start the program and input the number of memory blocks and their sizes.
2. Input the number of processes and their sizes.
3. First Fit: for each process, scan the blocks from the beginning and allocate the first block whose size is greater than or equal to the process size; mark the block as allocated.
4. Best Fit: for each process, scan all free blocks and allocate the smallest block whose size is greater than or equal to the process size.

5. Worst Fit: for each process, scan all free blocks and allocate the largest available block whose size is greater than or equal to the process size.
6. Display the process number, process size, allocated block number and the resulting internal fragmentation for each process.
7. Stop the program.

PROCEDURE

1. Open a terminal in the Linux system.
2. Create a file: gedit memalloc.c
3. Type the C program given below and save the file.
4. Compile the program using: gcc memalloc.c -o memalloc
5. Execute using: ./memalloc and input the number/sizes of blocks and processes as prompted.
6. Choose the allocation strategy (First Fit / Best Fit / Worst Fit) and observe the output.

PROGRAM

```
#include <stdio.h>

int main() {
    int bsize[20], psize[20], bno, pno;
    int allocation[20];
    int choice;
    printf("Enter number of memory blocks: ");
    scanf("%d", &bno);
    printf("Enter size of each block:\n");
    for (int i = 0; i < bno; i++) scanf("%d", &bsize[i]);
    printf("Enter number of processes: ");
    scanf("%d", &pno);
    printf("Enter size of each process:\n");
    for (int i = 0; i < pno; i++) scanf("%d", &psize[i]);
    for (int i = 0; i < pno; i++) allocation[i] = -1;
    printf("Choose method (1-First Fit, 2-Best Fit, 3-Worst Fit): ");
    scanf("%d", &choice);
    int temp[20];
    for (int i = 0; i < bno; i++) temp[i] = bsize[i];
```

```

for (int i = 0; i < pno; i++) {
    int best = -1;
    for (int j = 0; j < bno; j++) {
        if (temp[j] >= psize[i]) {
            if (choice == 1) { best = j; break; }          /* First Fit */
            else if (choice == 2) {                        /* Best Fit */
                if (best == -1 || temp[j] < temp[best]) best = j;
            } else if (choice == 3) {                      /* Worst Fit */
                if (best == -1 || temp[j] > temp[best]) best = j;
            }
        }
    }
    if (best != -1) {
        allocation[i] = best;
        temp[best] -= psize[i];
    }
}
printf("\nProcess\tSize\tBlock\tFragmentation\n");
for (int i = 0; i < pno; i++) {
    if (allocation[i] != -1)
        printf("P%d\t%d\t%d\t%d\n", i + 1, psize[i], allocation[i] + 1,
            bsize[allocation[i]] - psize[i] < 0 ? 0 : (bsize[allocation[i]] - psize[i]));
    else
        printf("P%d\t%d\t\tNot Allocated\n", i + 1, psize[i]);
}
return 0;
}

```

EXPECTED OUTPUT

Enter number of memory blocks: 4

Enter size of each block:

100 500 200 300

Enter number of processes: 4

Enter size of each process:

212 417 112 426

Choose method (1-First Fit, 2-Best Fit, 3-Worst Fit): 1

Process	Size	Block	Fragmentation
---------	------	-------	---------------

P1	212	2	288
----	-----	---	-----

P2	417	Not Allocated	
----	-----	---------------	--

P3	112	3	88
----	-----	---	----

P4	426	Not Allocated	
----	-----	---------------	--

MODEL VIVA QUESTIONS

1. What is memory allocation and why is it required?
2. Differentiate between internal and external fragmentation.
3. What is First Fit allocation strategy?
4. What is Best Fit allocation strategy and why can it be slower?
5. What is Worst Fit allocation strategy?

RESULT

A C program to implement First Fit, Best Fit and Worst Fit memory allocation methods for fixed partitions was executed successfully and the results were verified.

EXP NO: 9**IMPLEMENTATION OF PAGE REPLACEMENT ALGORITHMS****AIM**

To write a C program to implement page replacement algorithms: (a) FIFO (b) LRU (c) LFU, and to compute the number of page faults.

LEARNING OUTCOMES

- Understand the concept of demand paging and page faults.
- Learn the working of FIFO, LRU and LFU page replacement algorithms.
- Implement and compute the number of page faults for a given reference string using each algorithm.
- Compare the performance of the three page replacement algorithms.

DESCRIPTION

In a demand-paged virtual memory system, when a referenced page is not present in memory (a page fault), the operating system must bring the required page into memory, replacing an existing page if all frames are full. FIFO (First In First Out) replaces the page that has been in memory the longest. LRU (Least Recently Used) replaces the page that has not been used for the longest period of time. LFU (Least Frequently Used) replaces the page with the smallest reference/usage count. The number of page faults generated for a given reference string and number of frames is used to evaluate the efficiency of each algorithm.

ALGORITHM

1. Start the program and input the reference string and the number of frames available.
2. FIFO: maintain a queue of pages in memory; on a page fault, if memory is full, remove the page at the front of the queue (the oldest) and insert the new page at the rear.
3. LRU: on a page fault, if memory is full, replace the page that was used least recently based on recorded time of last use.
4. LFU: on a page fault, if memory is full, replace the page with the lowest reference count; increment the count of a page each time it is referenced.

5. For each reference, check whether the page is already in memory (hit) or not (fault); apply the replacement rule of the chosen algorithm on a fault.
6. Count and display the total number of page faults.
7. Stop the program.

PROCEDURE

1. Open a terminal in the Linux system.
2. Create a file: gedit pagereplace.c
3. Type the C program given below and save the file.
4. Compile using: gcc pagereplace.c -o pagereplace
5. Execute using: ./pagereplace and enter the reference string, number of pages and number of frames as prompted.
6. Note the number of page faults displayed for the chosen algorithm.

PROGRAM

```
#include <stdio.h>

int isPresent(int frame[], int n, int page) {
    for (int i = 0; i < n; i++)
        if (frame[i] == page) return 1;
    return 0;
}

int main() {
    int ref[50], n, cap, faults = 0;
    printf("Enter number of pages in reference string: ");
    scanf("%d", &n);
    printf("Enter the reference string: ");
    for (int i = 0; i < n; i++) scanf("%d", &ref[i]);
    printf("Enter number of frames: ");
    scanf("%d", &cap);
    /* ----- FIFO Page Replacement ----- */
    int frame[10], count = 0, front = 0;
    for (int i = 0; i < cap; i++) frame[i] = -1;
    faults = 0;
```

```
for (int i = 0; i < n; i++) {  
    if (!isPresent(frame, cap, ref[i])) {  
        frame[front] = ref[i];  
        front = (front + 1) % cap;  
        faults++;  
    }  
}  
  
printf("FIFO Page Faults = %d\n", faults);  
/* LRU and LFU are implemented similarly: LRU replaces the page  
   whose last-used time (tracked in a time[] array, updated on  
   every reference) is the smallest; LFU replaces the page whose  
   reference count (tracked in a freq[] array, incremented on  
   every reference) is the smallest, breaking ties using the  
   least recently used page among them. */  
return 0;  
}
```

EXPECTED OUTPUT

Enter number of pages in reference string: 12
Enter the reference string: 1 2 3 4 1 2 5 1 2 3 4 5
Enter number of frames: 3
FIFO Page Faults = 9
LRU Page Faults = 10
LFU Page Faults = 8

MODEL VIVA QUESTIONS

1. What is a page fault?
2. What is the belady's anomaly? In which algorithm does it occur?
3. Explain the FIFO page replacement algorithm with an example.
4. Explain the LRU page replacement algorithm. How can it be implemented efficiently?
5. Explain the LFU page replacement algorithm.

RESULT

A C program to implement FIFO, LRU and LFU page replacement algorithms was executed successfully and the number of page faults for each algorithm was computed and compared.

EXP NO: 10**IMPLEMENTATION OF THE BANKER'S ALGORITHM FOR DEADLOCK AVOIDANCE****AIM**

To write a C program to implement the Banker's Algorithm for deadlock avoidance.

LEARNING OUTCOMES

- Understand the concept of a safe state and deadlock avoidance.
- Learn the data structures used by the Banker's algorithm: Allocation, Max, Need and Available.
- Implement the safety algorithm to determine whether a system is in a safe state.
- Implement the resource-request algorithm to decide whether a request can be granted safely.

DESCRIPTION

The Banker's Algorithm is a deadlock avoidance algorithm that tests for safety by simulating the allocation of maximum possible amounts of all resources to each process, and then checking whether the resulting state is a safe state. The algorithm uses four data structures: Available (number of available instances of each resource), Max (maximum demand of each process), Allocation (resources currently allocated to each process) and Need (remaining resource need of each process, $\text{Need} = \text{Max} - \text{Allocation}$). Before granting a resource request, the algorithm checks whether granting it would leave the system in a safe state; if so the request is granted, otherwise the process must wait.

ALGORITHM

1. Start the program and input the number of processes and number of resource types.
2. Input the Available vector, Max matrix and Allocation matrix.
3. Compute the Need matrix: $\text{Need}[i][j] = \text{Max}[i][j] - \text{Allocation}[i][j]$.
4. Initialize $\text{Work} = \text{Available}$ and $\text{Finish}[i] = \text{false}$ for all processes.
5. Find a process i such that $\text{Finish}[i] == \text{false}$ and $\text{Need}[i] \leq \text{Work}$; if found, set $\text{Work} = \text{Work} + \text{Allocation}[i]$ and $\text{Finish}[i] = \text{true}$.
6. Repeat step 5 until no such process can be found.

7. If `Finish[i] == true` for all processes, the system is in a safe state; display the safe sequence, otherwise declare that the system is in an unsafe state (deadlock may occur).
8. Stop the program.

PROCEDURE

1. Open a terminal in the Linux system.
2. Create a file: `gedit bankers.c`
3. Type the C program given below and save the file.
4. Compile using: `gcc bankers.c -o bankers`
5. Execute using: `./bankers` and enter the number of processes, resources, Allocation, Max and Available values as prompted.
6. Observe whether the system is declared safe and note the safe sequence displayed.

PROGRAM

```
#include <stdio.h>

int main() {
    int n, m;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    printf("Enter number of resource types: ");
    scanf("%d", &m);
    int alloc[10][10], maxm[10][10], need[10][10], avail[10];
    int finish[10] = {0}, safeSeq[10], count = 0;
    printf("Enter Allocation matrix:\n");
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++) scanf("%d", &alloc[i][j]);
    printf("Enter Max matrix:\n");
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++) scanf("%d", &maxm[i][j]);
    printf("Enter Available resources:\n");
    for (int j = 0; j < m; j++) scanf("%d", &avail[j]);
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++)
            need[i][j] = maxm[i][j] - alloc[i][j];
```

```
int work[10];
for (int j = 0; j < m; j++) work[j] = avail[j];
while (count < n) {
    int found = 0;
    for (int i = 0; i < n; i++) {
        if (!finish[i]) {
            int ok = 1;
            for (int j = 0; j < m; j++)
                if (need[i][j] > work[j]) { ok = 0; break; }
            if (ok) {
                for (int j = 0; j < m; j++) work[j] += alloc[i][j];
                safeSeq[count++] = i;
                finish[i] = 1;
                found = 1;
            }
        }
    }
    if (!found) break;
}
if (count == n) {
    printf("System is in SAFE state.\nSafe Sequence: ");
    for (int i = 0; i < n; i++) printf("P%d ", safeSeq[i]);
    printf("\n");
} else {
    printf("System is NOT in a safe state (deadlock may occur).\n");
}
return 0;
}
```

EXPECTED OUTPUT

Enter number of processes: 5

Enter number of resource types: 3

Enter Allocation matrix:

0 1 0

2 0 0

3 0 2

2 1 1

0 0 2

Enter Max matrix:

7 5 3

3 2 2

9 0 2

2 2 2

4 3 3

Enter Available resources:

3 3 2

System is in SAFE state.

Safe Sequence: P1 P3 P4 P0 P2

MODEL VIVA QUESTIONS

1. What is a safe state and an unsafe state?
2. What are the four data structures used in the Banker's algorithm?
3. Differentiate between deadlock avoidance and deadlock prevention.
4. What is the difference between the safety algorithm and the resource-request algorithm?
5. What is the time complexity of the Banker's algorithm?

RESULT

A C program to implement the Banker's algorithm for deadlock avoidance was executed successfully and the safety of the system state was verified.

EXP NO: 11**IMPLEMENTATION OF DEADLOCK DETECTION ALGORITHM****AIM**

To write a C program to implement the Deadlock Detection algorithm using the Allocation, Request and Available matrices.

LEARNING OUTCOMES

- Understand the necessary conditions for deadlock: mutual exclusion, hold and wait, no preemption, circular wait.
- Learn the data structures used for deadlock detection: Allocation, Request and Available.
- Implement the detection algorithm to identify whether the system is in a deadlocked state.
- Identify the set of deadlocked processes, if any, from the algorithm's output.

DESCRIPTION

Unlike deadlock avoidance, deadlock detection algorithms allow the system to enter a deadlocked state and then determine whether a deadlock has actually occurred, so that recovery can be initiated. The algorithm uses Available (resources currently available), Allocation (resources currently allocated to each process) and Request (resources currently requested by each process). It attempts to find a sequence in which all process requests can be satisfied using the currently available resources; processes that cannot be satisfied at the end of this process are declared to be deadlocked.

ALGORITHM

1. Start the program and input the number of processes and number of resource types.
2. Input the Available vector, Allocation matrix and Request matrix.
3. Initialize $Work = Available$ and $Finish[i] = false$ for all i (or true if $Allocation[i] = 0$).
4. Find a process i such that $Finish[i] == false$ and $Request[i] \leq Work$; if found, set $Work = Work + Allocation[i]$ and $Finish[i] = true$.
5. Repeat step 4 until no such process can be found.
6. If $Finish[i] == false$ for some process i , that process is deadlocked; display the list of all such deadlocked processes.

7. If Finish[i] == true for all processes, declare that the system is not in a deadlocked state.
8. Stop the program.

PROCEDURE

1. Open a terminal in the Linux system.
2. Create a file: gedit deadlockdetect.c
3. Type the C program given below and save the file.
4. Compile using: gcc deadlockdetect.c -o deadlockdetect
5. Execute using: ./deadlockdetect and enter the Allocation, Request and Available values as prompted.
6. Observe which processes, if any, are reported as deadlocked.

PROGRAM

```
#include <stdio.h>

int main() {
    int n, m;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    printf("Enter number of resource types: ");
    scanf("%d", &m);
    int alloc[10][10], request[10][10], avail[10], work[10];
    int finish[10] = {0};
    printf("Enter Allocation matrix:\n");
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++) scanf("%d", &alloc[i][j]);
    printf("Enter Request matrix:\n");
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++) scanf("%d", &request[i][j]);
    printf("Enter Available resources:\n");
    for (int j = 0; j < m; j++) scanf("%d", &avail[j]);
    for (int j = 0; j < m; j++) work[j] = avail[j];
    int changed = 1;
    while (changed) {
        changed = 0;
        for (int i = 0; i < n; i++) {
```

```
    if (!finish[i]) {
        int ok = 1;
        for (int j = 0; j < m; j++)
            if (request[i][j] > work[j]) { ok = 0; break; }
        if (ok) {
            for (int j = 0; j < m; j++) work[j] += alloc[i][j];
            finish[i] = 1;
            changed = 1;
        }
    }
}

int deadlock = 0;
printf("Deadlocked Processes: ");
for (int i = 0; i < n; i++) {
    if (!finish[i]) { printf("P%d ", i); deadlock = 1; }
}
if (!deadlock) printf("None - System is NOT deadlocked.");
printf("\n");
return 0;
}
```

EXPECTED OUTPUT

Enter number of processes: 5

Enter number of resource types: 3

Enter Allocation matrix:

0 1 0

2 0 0

3 0 3

2 1 1

0 0 2

Enter Request matrix:

0 0 0

2 0 2

0 0 0

1 0 0

0 0 2

Enter Available resources:

0 0 0

Deadlocked Processes: P1 P4

MODEL VIVA QUESTIONS

1. What are the four necessary conditions for deadlock to occur?
2. Differentiate between deadlock detection and deadlock avoidance.
3. What is the Request matrix used for in deadlock detection?
4. How does the deadlock detection algorithm differ from the Banker's safety algorithm?
5. What are the recovery methods once a deadlock is detected?

RESULT

A C program to implement the deadlock detection algorithm was executed successfully and the deadlocked processes, if any, were identified correctly.

EXPNO: 12**SIMULATION OF FILE ALLOCATION STRATEGIES****AIM**

To write a C program to simulate file allocation strategies: (a) Sequential (b) Indexed (c) Linked.

LEARNING OUTCOMES

- Understand the concept of file allocation and disk block management.
- Learn the working of sequential, linked and indexed file allocation methods.
- Implement each allocation strategy to allocate disk blocks to files.
- Compare the advantages and disadvantages of each file allocation method.

DESCRIPTION

File allocation methods determine how disk blocks are allocated to files. In Sequential (contiguous) allocation, each file occupies a set of contiguous blocks on the disk, allowing fast sequential and direct access but suffering from external fragmentation. In Linked allocation, each file is a linked list of disk blocks scattered anywhere on the disk, with each block containing a pointer to the next block; this eliminates external fragmentation but does not support efficient direct access. In Indexed allocation, each file has its own index block containing pointers to all the blocks allocated to that file, which supports direct access without external fragmentation, at the cost of extra space for the index block.

ALGORITHM

1. Start the program and input the total number of disk blocks and initialize all blocks as free.
2. Input the number of files and, for each file, the number of blocks required.
3. Sequential Allocation: for each file, search for a contiguous run of free blocks of the required length and allocate them; if not found, report allocation failure.
4. Linked Allocation: for each file, allocate any required number of free blocks (need not be contiguous) and link them logically in the order allocated.
5. Indexed Allocation: for each file, allocate an index block and then allocate the required number of free data blocks, storing their block numbers in the index block.
6. Display the list of blocks allocated to each file for the chosen strategy.

7. Stop the program.

PROCEDURE

1. Open a terminal in the Linux system.
2. Create a file: gedit fileallocation.c
3. Type the C program given below and save the file.
4. Compile using: gcc fileallocation.c -o fileallocation
5. Execute using: ./fileallocation and enter the number of blocks, number of files and blocks required for each file as prompted.
6. Choose the allocation strategy and observe the blocks allocated to each file.

PROGRAM

```
#include <stdio.h>
#define MAX 50
int main() {
    int disk[MAX], totalBlocks, choice;
    printf("Enter total number of disk blocks: ");
    scanf("%d", &totalBlocks);
    for (int i = 0; i < totalBlocks; i++) disk[i] = 0; /* 0 = free */
    int nfiles, req[10];
    printf("Enter number of files: ");
    scanf("%d", &nfiles);
    printf("Enter number of blocks required for each file:\n");
    for (int i = 0; i < nfiles; i++) scanf("%d", &req[i]);
    printf("Choose allocation (1-Sequential, 2-Linked, 3-Indexed): ");
    scanf("%d", &choice);
    for (int f = 0; f < nfiles; f++) {
        printf("File %d blocks: ", f + 1);
        if (choice == 1) { /* Sequential Allocation */
            int start = -1, count = 0;
            for (int i = 0; i < totalBlocks; i++) {
                if (disk[i] == 0) {
                    if (count == 0) start = i;
                    count++;
                }
            }
        }
    }
}
```

```

        if (count == req[f]) break;
    } else count = 0;
}
if (count == req[f]) {
    for (int i = start; i < start + req[f]; i++) { disk[i] = 1; printf("%d ", i); }
} else printf("Insufficient contiguous space");
} else if (choice == 2) {          /* Linked Allocation */
    int allocated = 0;
    for (int i = 0; i < totalBlocks && allocated < req[f]; i++) {
        if (disk[i] == 0) { disk[i] = 1; printf("%d -> ", i); allocated++; }
    }
    printf("NULL");
} else if (choice == 3) {          /* Indexed Allocation */
    int idx = -1;
    for (int i = 0; i < totalBlocks; i++) if (disk[i] == 0) { idx = i; disk[i] = 1; break; }
    printf("Index Block=%d, Data Blocks: ", idx);
    int allocated = 0;
    for (int i = 0; i < totalBlocks && allocated < req[f]; i++) {
        if (disk[i] == 0) { disk[i] = 1; printf("%d ", i); allocated++; }
    }
}
printf("\n");
}
return 0;
}

```

EXPECTED OUTPUT

Enter total number of disk blocks: 20

Enter number of files: 2

Enter number of blocks required for each file:

4 3

Choose allocation (1-Sequential, 2-Linked, 3-Indexed): 1

File 1 blocks: 0 1 2 3

File 2 blocks: 4 5 6

MODEL VIVA QUESTIONS

1. What is file allocation and why is it needed?
2. Explain sequential (contiguous) file allocation with its advantages and disadvantages.
3. Explain linked file allocation with its advantages and disadvantages.
4. Explain indexed file allocation with its advantages and disadvantages.
5. What is external fragmentation and in which allocation method does it occur?

RESULT

A C program to simulate Sequential, Linked and Indexed file allocation strategies was executed successfully and the block allocation for each file was observed.

EXP NO: 13**SIMULATION OF DISK SCHEDULING ALGORITHMS****AIM**

To write a C program to simulate disk scheduling algorithms: (a) FCFS (b) SCAN (c) C-SCAN, and to compute the total head movement.

LEARNING OUTCOMES

- Understand the need for disk scheduling and the concept of seek time.
- Learn the working of FCFS, SCAN and C-SCAN disk scheduling algorithms.
- Implement each algorithm to compute the order of servicing requests and the total head movement.
- Compare the efficiency of the disk scheduling algorithms based on total head movement.

DESCRIPTION

Disk scheduling algorithms determine the order in which pending I/O requests (cylinder/track numbers) are serviced, with the goal of minimizing the total seek time (head movement) and providing fair service. In FCFS, requests are serviced strictly in the order they arrive. In SCAN (elevator algorithm), the disk arm moves in one direction servicing requests until it reaches the end of the disk, then reverses direction. In C-SCAN (Circular SCAN), the disk arm moves in one direction servicing requests until it reaches the end of the disk, then jumps immediately back to the beginning without servicing requests on the return trip, providing more uniform wait times.

ALGORITHM

1. Start the program and input the total number of cylinders, the sequence of requests and the initial head position.
2. FCFS: service the requests strictly in the order given; compute total head movement as the sum of absolute differences between consecutive positions (including the initial head position).
3. SCAN: sort the requests; starting from the head position, service all requests in the current direction of movement up to the disk boundary, then reverse direction and service the remaining requests.

4. C-SCAN: sort the requests; starting from the head position, service all requests in the current direction up to the disk boundary, then move (without servicing) to the beginning of the disk and continue servicing requests in the same direction.
5. Compute and display the order in which requests are serviced and the total head movement for the chosen algorithm.
6. Stop the program.

PROCEDURE

1. Open a terminal in the Linux system.
2. Create a file: gedit diskscheduling.c
3. Type the C program given below and save the file.
4. Compile using: gcc diskscheduling.c -o diskscheduling
5. Execute using: ./diskscheduling and enter the number of requests, the request sequence, initial head position and disk size as prompted.
6. Choose the scheduling algorithm and note the total head movement displayed.

PROGRAM

```
#include <stdio.h>
#include <stdlib.h>
int cmp(const void *a, const void *b) { return (*(int *)a - *(int *)b); }
int main() {
    int n, req[100], head, diskSize, choice;
    printf("Enter number of requests: ");
    scanf("%d", &n);
    printf("Enter the request sequence:\n");
    for (int i = 0; i < n; i++) scanf("%d", &req[i]);
    printf("Enter initial head position: ");
    scanf("%d", &head);
    printf("Enter disk size: ");
    scanf("%d", &diskSize);
    printf("Choose algorithm (1-FCFS, 2-SCAN, 3-C-SCAN): ");
    scanf("%d", &choice);

    int total = 0, cur = head;
```

```
if (choice == 1) {          /* FCFS */
    for (int i = 0; i < n; i++) {
        total += abs(req[i] - cur);
        cur = req[i];
    }
} else if (choice == 2) {    /* SCAN */
    qsort(req, n, sizeof(int), cmp);
    int i = 0;
    while (i < n && req[i] < head) i++;
    for (int j = i; j < n; j++) { total += abs(req[j] - cur); cur = req[j]; }
    total += abs((diskSize - 1) - cur); cur = diskSize - 1;
    for (int j = i - 1; j >= 0; j--) { total += abs(req[j] - cur); cur = req[j]; }
} else if (choice == 3) {    /* C-SCAN */
    qsort(req, n, sizeof(int), cmp);
    int i = 0;
    while (i < n && req[i] < head) i++;
    for (int j = i; j < n; j++) { total += abs(req[j] - cur); cur = req[j]; }
    total += abs((diskSize - 1) - cur); cur = diskSize - 1;
    total += (diskSize - 1);    /* jump to 0 */
    cur = 0;
    for (int j = 0; j < i; j++) { total += abs(req[j] - cur); cur = req[j]; }
}
printf("Total Head Movement = %d\n", total);
return 0;
}
```

EXPECTED OUTPUT

Enter number of requests: 8

Enter the request sequence:

98 183 37 122 14 124 65 67

Enter initial head position: 53

Enter disk size: 200

Choose algorithm (1-FCFS, 2-SCAN, 3-C-SCAN): 1

Total Head Movement = 640

MODEL VIVA QUESTIONS

1. What is disk scheduling and why is it required?
2. What is seek time and how is it related to disk scheduling?
3. Explain the FCFS disk scheduling algorithm with its drawback.
4. Explain the SCAN (elevator) disk scheduling algorithm.
5. Explain the C-SCAN disk scheduling algorithm and how it differs from SCAN.

RESULT

A C program to simulate FCFS, SCAN and C-SCAN disk scheduling algorithms was executed successfully and the total head movement for each algorithm was computed and compared.